

Recovering Semantics from Unknown Binaries with Known Input Formats

Anonymous
Anonymous Institution

Abstract—Reverse engineers who already understand the input format a binary parses spend most of their initial effort locating the code that implements each piece of the format. We present ParseRE, a binary-only tool that automates that step. Given a grammar template for the input format, ParseRE instantiates a corpus of structurally varied inputs, traces each one under QEMU user-mode emulation, compares pairs of traces whose parse trees differ in exactly one production, and attributes the differential basic blocks to that production. A two-step uniqueness filter removes shared code, and the result is a labeled control-flow graph in which most parser-relevant basic blocks carry the name of the grammar rule they implement. Unlike prior approaches, ParseRE requires no source code, no compiler instrumentation, and no data-flow analysis. We evaluate on parsers for four formats spanning the language hierarchy: URI (regular), JSON (context-free), HPACK (context-sensitive binary), and ELF (offset-driven binary). On the completed ELF evaluation against libelf, manual inspection of every label yields 91.2% strict accuracy across 68 labeled basic blocks. [BEN: URI and JSON accuracy rates pending Rishav’s labeled SVG handoff. HPACK numbers pending your evaluation.]

Index Terms—reverse engineering, binary analysis, control-flow graphs, parser analysis, input format recovery, QEMU

1. Introduction

Modern reverse-engineering workflows begin with a binary, a vague sense of what it does, and a long road from there to understanding. When the binary parses a known input format, much of that road can be skipped: any reverse engineer who already understands the format spends most of their initial effort locating the code that implements each piece of it. Existing tools either annotate the input (PolyFile [1]), require source-level instrumentation to track data flow through the parser (PolyTracker [2]), or compare two execution traces by hand (Tenet [3]). None of them produce a labeled map of the binary’s control-flow graph from a known grammar alone.

The Core Idea. We present ParseRE, a binary-only labeling tool driven by parse-tree differencing. Given a grammar template for the target’s input format,

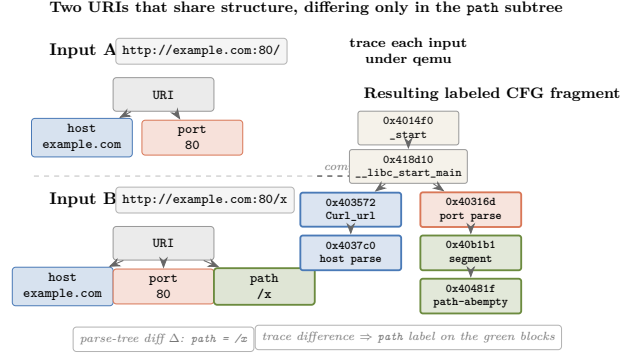


Figure 1. Two URIs that share scheme, host, and port but differ in the path subtree. The trace through the parser differs in a small set of basic blocks; those blocks receive the path label. ParseRE generalizes this idea over a corpus of structurally varied inputs.

ParseRE instantiates a corpus of structurally varied inputs, traces each one under QEMU user-mode emulation, and compares pairs of traces whose parse trees differ in exactly one production. Figure 1 sketches the idea on two URIs that differ only in their path subtree: the trace blocks that fire for Input B but not Input A get attributed to the path production. Aggregated over the full corpus and filtered for uniqueness, the result is a labeled control-flow graph in which most parser-relevant basic blocks carry the name of the grammar rule they implement.

Why this works in practice. Parsers tend to dispatch on grammar production at the function level. A URI parser has a separate routine for host validation than for path canonicalization. A JSON parser has separate code paths for numeric tokens, string tokens, and structural tokens. Two inputs that differ only in their path subtree therefore tend to differ only in the basic blocks that process paths. ParseRE exploits this regularity directly: the set of edges present in one trace but not the other tracks the production that varied between them.

Why this is hard in practice. Real parsers also share large amounts of code across productions, including memory allocation, error reporting, and common token tape advancement. Naively attributing every differential edge to a production produces a noisy label set in which most labels appear on most blocks. ParseRE applies a

two-step uniqueness filter that strips edges shared across multiple productions before reporting a label, which is the difference between unusable output and the 91.2% strict accuracy we measure on libelf.

Evaluation. We evaluate ParseRE on four input formats spanning the Chomsky hierarchy [4]: URI (regular, parsed by Curl), JSON (context-free, parsed by json-c), HPACK (context-sensitive binary, parsed by nghttp2), and ELF (offset-driven binary, parsed by libelf). For each format we run ParseRE on a real-world parser and manually inspect every labeled basic block. The ELF evaluation, which is fully labeled, yields 68 labeled blocks at 91.2% accuracy. [BEN: HPACK numbers pending your run. RISHAV: URI/JSON per-label TP/FP/FN pending your handoff.]

Contributions.

- We introduce the first binary-only technique that transfers semantic labels from program inputs onto control-flow graph nodes and edges (Section 3).
- We implement the technique in ParseRE, a Python tool that requires only the target binary and a grammar template, and release it as open source.[BEN: confirm public URL for camera-ready.]
- We evaluate ParseRE on real parsers (Curl, json-c, libelf, nghttp2, Apache APR) and manually inspect every label produced (Section 5).
- We identify a structural property of binary parsers (uniform record readers) that explains why binary formats produce fewer but higher-confidence labels than text formats, and show that ParseRE captures this property without prior knowledge of it.

2. Background

2.1. Control-Flow Graphs

Reverse engineering is the process of analyzing a binary to understand its structure, functionality, and behavior without access to source code. A fundamental program representation in this domain is the control-flow graph (CFG). Each node in a CFG corresponds to a basic block: a maximal sequence of consecutive instructions with a single entry point and a single exit point [5]. If execution reaches the first instruction of a basic block, it will execute every instruction in that block before transferring control elsewhere. Edges in a CFG represent transfers of control flow, including branches, function calls, and returns.

Understanding how a program’s input drives its execution path through the CFG is crucial to understanding the program’s behavior. For a parser binary, different inputs exercise different paths, and the structural differences between inputs correspond to structural differences in the paths taken through the parser code.

2.2. Execution Tracing with QEMU

QEMU [6] is an open-source emulator that supports both full-system and user-mode emulation across many architectures. In user-mode emulation, QEMU executes a single Linux binary by translating its instructions and forwarding system calls to the host kernel.

A useful feature for binary analysis is QEMU’s logging facility, which records every basic block the emulator translates along with the block’s start address and the disassembly of its first and last instructions. This produces a complete, ordered basic-block trace of one program execution without any modification to the target binary.

This capability is central to our approach. We trace the target under QEMU for many inputs and compare the resulting traces to infer which code regions correspond to which input features.

2.3. Parse Trees and Input Annotations

A parse tree represents the syntactic structure of an input according to a grammar. Internal nodes correspond to grammar productions; leaves hold the concrete bytes.

We use parse trees to annotate inputs: each subtree is labeled with the grammar production it instantiates. When two inputs share the same tree structure except at one subtree, the difference in their execution traces can be attributed to the differing production. This is the core insight behind ParseRE.

3. Design

ParseRE runs as a four-stage pipeline, shown in Figure 2. The top row converts a grammar template into traced execution. The bottom row converts traced execution into a labeled CFG. Every stage except the grammar template itself is format-agnostic; adding a new format requires only writing a new template and compiling a small harness against the parser library under test.

3.1. Corpus Generation

A grammar template is a tree of production rules. Each leaf carries a small set of byte-level alternatives; each internal node carries a set of child configurations. ParseRE enumerates every combination of alternatives through Cartesian product, producing a corpus of concrete inputs each paired with its parse tree.

The templates are deliberately small. The JSON template covers objects, arrays, strings, numbers, booleans, null, and a handful of degenerate cases (nan, inf), instantiating to 27 inputs. The URI template covers scheme, authority, path, query, and fragment, instantiating to 1,458 inputs. The ELF template, described in Section 5.5, instantiates to 20 inputs.

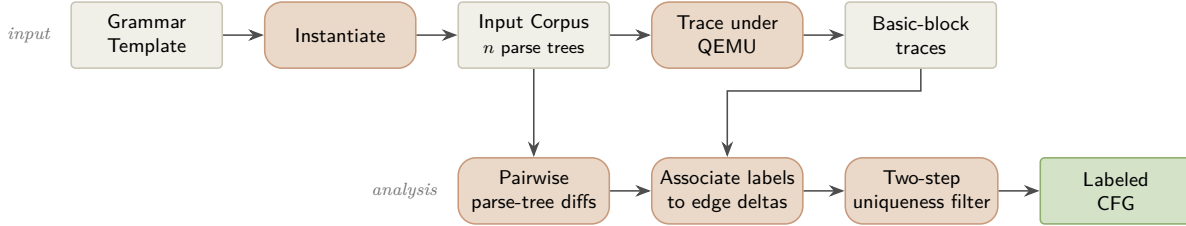


Figure 2. The ParseRE pipeline. Cream rectangles are data artifacts. Salmon rounded rectangles are processing steps. The green rectangle is the final output. Vertical arrows bridge the two rows: the input corpus drives both the tracing pass (top) and the pairwise diff computation (bottom), and the resulting basic-block traces feed the label-association step.

The key property. For every pair of inputs we generate, the parse trees differ in either a single production or a small set of productions. This is what makes attribution possible. If two inputs differed in arbitrary ways, the trace differences between them could not be cleanly attributed to any single grammar element.

3.2. Execution Tracing

Each input is fed to the target binary, which runs under QEMU in user-mode emulation. QEMU logs every translated basic block to a per-input trace file with the block’s start address and the disassembly of its first and last instructions. ParseRE parses each trace into an ordered block sequence, retains only blocks within the target binary’s executable segment, and converts the sequence into an edge map: the set of consecutive basic-block pairs observed during execution.

Traces from inputs that the parser rejects (non-zero exit status) are dropped. The CFG is built from successful parses, where the parser exercises its full grammar-handling logic.

3.3. Label Association

For every pair of inputs (T_i, T_j) whose parse trees differ in exactly one production p , ParseRE computes the edge delta $E_i \setminus E_j$ and attributes those edges to p . Aggregating over every input pair that isolates p yields the per-production edge set.

Two-step uniqueness filter. Raw edge-to-label associations are noisy. Many edges are shared across productions, including error-handling code and shared token-tape advancement. ParseRE strips this noise in two steps:

- 1) Subset removal. If the edge set for production p_1 is nearly a subset of the edge set for p_2 ($|p_1 \setminus p_2|/|p_1| < 0.1$), ParseRE removes p_1 ’s edges from p_2 ’s set. This eliminates setup and teardown code that runs for multiple productions but is most strongly associated with one.
- 2) Strict uniqueness. For every production, ParseRE subtracts the edges attributed to every

other production. Edges that survive belong to a single production and receive its label.

After filtering, ParseRE propagates colors backward through the CFG. A node whose outgoing edges all carry the same label is itself labeled with that color, which captures basic blocks that serve as sole entry points into a production-specific subgraph.

3.4. Output

ParseRE emits three artifacts: a Graphviz DOT file of the labeled CFG, a rendered SVG for browser inspection, and an address-to-label mapping the analyst can import into a disassembler. A companion Ghidra script consumes the address mapping and attaches per-block annotations to the disassembly listing, which lets the analyst inspect labels directly inside the reverse-engineering workflow.

4. Implementation

ParseRE is implemented as a single Python module of around 850 lines, with five external dependencies (networkx, pydot, pyelftools, tqdm, cxxfilt) and a parallel work pool that scales to all available cores. Running ParseRE requires Python 3.13 and a QEMU user-mode build for the target architecture. The analyst supplies the target binary, a grammar template for the input format, and optionally a path to addr2line for source-line attribution during manual evaluation.

4.1. Corpus Generation

Grammar templates are written as nested data structures. Each template node carries a production name and a set of alternatives. At leaves, alternatives are concrete byte sequences. At internal nodes, alternatives are ordered tuples of child templates. ParseRE recursively enumerates every combination through Cartesian product, producing a set of concrete parse trees that serialize to byte strings.

Branching factor sets corpus size. The JSON template yields 27 instantiations (729 pairwise comparisons), the URI template yields 1,458 instantiations

(over two million pairwise comparisons), and the ELF template yields 20 instantiations (380 pairwise comparisons). The text-format templates are larger because the underlying grammars have more independently varying productions. Each input remains small (under one kilobyte for the text formats, exactly 1,048 bytes for ELF), which keeps trace collection practical even when the input count grows.

Adding a new format. A new format requires two pieces: a template definition and a harness binary that exercises the parser library under test. The template plugs into the existing format dispatch. The harness compiles into the Docker image alongside the existing harnesses. No changes to the core pipeline are needed.

4.2. Tracing with QEMU

Each input is fed to the target via standard input. ParseRE invokes QEMU with the basic-block logging flag enabled and a per-input output path, parses the resulting log with a regular expression that matches QEMU’s basic-block records, and constructs a basic-block object per record. Only blocks within the target binary’s executable segment are retained; blocks from the dynamic linker and the QEMU runtime itself are discarded by checking each address against the target’s `PT_LOAD` program header.

Traces for inputs with non-zero exit status are excluded. This ensures that the CFG is built from successful parses, where the parser exercises its full grammar-handling logic.

[BEN: the basic-block log format depends on the QEMU version. The current regex was written against QEMU 8.x as packaged on Debian bookworm. Worth pinning the version in the artifact description, or adding a fallback regex for older versions.]

4.3. Target Harnesses

For each parser under test we compile a small C harness that links against the parser library, reads input from standard input, invokes the top-level parse function, and returns the result. Harnesses are typically twenty to thirty lines. The ELF harness is longer (around one hundred lines) because libelf exposes the parser as a family of small calls (one per program header, one per section header, one per symbol) rather than a single top-level entry point, and the harness must call all of them to exercise the full structure.

Static linking is required. Dynamic linking caused the first URL evaluation to produce zero labels: the dynamic linker startup ran on every input, dominated every trace, and ParseRE’s uniqueness filter consequently wiped the entire graph as shared code. Rebuilding curl from source with `--without-shared` produced a static archive that the harness could link against, which restored the trace differential signal. Every harness in

the current evaluation links statically (libelf via the `-static` GCC flag, json-c via its `.a` archive from a CMake build, curl via its source-built static archive).

Compilation flags. All harnesses compile with `-O0 -fno-inline` so the compiler does not collapse distinct parser functions into shared inlined code, which would destroy the correspondence between grammar productions and basic-block ranges. We also use `-no-pie` to keep addresses stable so that `addr2line` resolves cleanly against the binary on disk, and `-g` for debug symbols.

4.4. Supported Formats

The current implementation includes grammar templates for four formats:

- URI (regular): scheme, authority (userinfo, host, port), path, query, fragment, per RFC 3986.
- JSON (context-free): objects, arrays, strings, numbers, booleans, null, per RFC 8259.
- HPACK (context-sensitive binary): indexed headers, literal headers with and without Huffman compression, per RFC 7541. [BEN: describe your template choices here when ready.]
- ELF (offset-driven binary): one fixed ELF header, four program-header variants, five section variants. Each instantiation is a valid 1,048-byte ELF64 file constructed by a Python generator that keeps the inter-field offsets consistent across all twenty corpus members.

5. Evaluation

We evaluate ParseRE on four input formats with at least one open-source parser implementation each. For every format-parser pair, we manually inspect every labeled basic block to determine whether the label is correct.

Research questions.

- (Q1) How many basic blocks does ParseRE label, and what fraction of the parser CFG do those labels cover?
- (Q2) How accurate are the labels at the per-block level?
- (Q3) Does the labeling behave consistently across language classes (regular, context-free, context-sensitive binary, offset-driven binary)?

5.1. Methodology

For each format-parser pair, we (1) compile a minimal harness against the parser library inside a Docker container with debug symbols, (2) run ParseRE with the corresponding grammar template under QEMU user-mode, and (3) manually inspect the output. For every labeled edge and node, a human evaluator checks whether the source location (recovered via `addr2line`)

implements the grammar production indicated by the label, and records true positives, false positives, and false negatives.

Why manual inspection. A previous iteration of this work used a large language model for label verification and achieved roughly 50% accuracy. We treat that as a baseline for what an automated verifier can do without semantic grounding in the target binary, and we use manual inspection instead. The numbers reported below are all from human evaluators reading the labeled SVG output and the disassembly of each labeled block.

5.2. URI Parsers

We evaluate on two URI parsers: Curl’s `curl_url_set` and Apache Portable Runtime’s `apr_uri_parse`. The URI template instantiates 1,458 inputs and produces 2,125,764 pairwise comparisons.

Curl. On the static-linked Curl harness, ParseRE produces a labeled CFG with 522 nodes and 558 edges, of which 211 basic blocks receive labels across 11 URI productions. Table 1 shows the distribution. Host validation dominates because Curl’s parser performs extensive checks: IPv4 versus IPv6 detection, IDNA processing, bracket handling, and length validation. The duplicate-looking labels (port versus :port, query versus ?query) come from the template’s leaf naming; the colon or question mark delimiter is included in the label string. This is a template quirk; the underlying attribution is correct.

TABLE 1. URI label distribution (Curl).

Production	Labeled BBs
host	91
port	34
segment	30
userinfo	25
query	5
fragment	5
path	1
ambiguous	21

[RISHAV: drop in TP/FP/FN counts here once your labeled SVG handoff is ready. The 21 ambiguous blocks are shared code paths ParseRE could not uniquely attribute to a single production.]

Apache APR. [BEN: same URI template, different parser. Rishav was going to run this. Numbers pending.]

5.3. JSON Parsers

We evaluate on `json-c`, a widely deployed C library for JSON parsing. The JSON template instantiates 27 inputs and produces 729 pairwise comparisons.

On `json-c` (commit 894856), ParseRE produces a labeled CFG with 464 nodes and 400 edges, of which 294 basic blocks receive labels across 6 productions. Table 2 shows the distribution.

TABLE 2. JSON label distribution (`json-c`).

Production	Labeled BBs
number	110
object-element	77
string	39
array	36
boolean	24
nan	9

number dominates because C-level numeric parsing is complex: sign handling, decimal point, exponent notation, overflow checks, and strtod wrappers each require their own basic blocks. nan receives few labels because the parsing path is a three-character string comparison.

[RISHAV: JSON TP/FP/FN counts also pending. Expect some shared error-handling blocks attributed to the wrong production.]

5.4. HPACK Decoder

HPACK is the header compression format for HTTP/2, defined in RFC 7541 [7]. The format is fully binary, uses context-sensitive encoding, and includes a fixed Huffman table baked into the specification. This makes HPACK a stress test for ParseRE: it has the offset-driven properties of ELF together with conditional encoding logic that text formats lack.

[BEN: drop your HPACK results here when ready. Target is nghttp2’s HPACK decoder. Expected productions: `indexed_header`, `literal_header_name_index`, `literal_header_both_strings`, `huffman_string`, `raw_string`, `variable_length_int`.]

5.5. ELF Parser

ELF is the standard binary format on Linux and other Unix-like systems. Unlike the sequential text formats above, ELF is offset-driven: the ELF header contains pointers to program header and section header tables, which in turn point to string tables, symbol tables, and other sections. We target libelf from elfutils, the reference library used by Ghidra, radare2, and most binary analysis toolchains.

Corpus layout. Figure 3 shows the layout of a single corpus instantiation. Every corpus member is exactly 1,048 bytes with identical offsets, and varies only in three regions: the ELF header (one variant), the two-entry program header table (four variants that change the second program header’s type field across `PT_NOTE`, `PT_INTERP`, `PT_DYNAMIC`, and `PT_NULL`), and the section region (five variants that toggle which sections contain real data versus zeroed bytes). The Cartesian product yields $1 \times 4 \times 5 = 20$ instantiations and 380 pairwise comparisons.

Results. On libelf, ParseRE produces a labeled CFG with 358 nodes and 354 edges. Of these, 68 basic blocks

One 1048-byte ELF64 file. Offsets fixed across all 20 corpus instantiations.

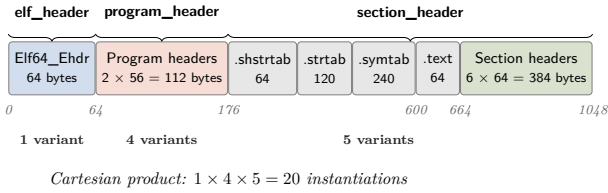


Figure 3. Fixed-layout ELF corpus. Every one of the 20 instantiations has the same byte offsets. The three template children (elf_header, program_header, section_header) carry 1, 4, and 5 alternatives respectively.

receive the section_header label. The program_header production receives zero labels despite having four template variants.

Why program_header is unlabeled. Libelf reads all program-header types through the same code path. The p_type field is data stored in a struct; libelf does not branch on it during parsing. The basic-block trace through gelf_getphdr is therefore identical across the four variants, and the difference algorithm has no edges to attribute. Section types, by contrast, do trigger different code paths: sections with SHT_SYMTAB type cause the harness to iterate symbol entries via gelf_getsym and resolve names via elf_strptr, while null sections skip these operations.

TABLE 3. ELF label distribution (libelf). The high ambiguity rate reflects libelf’s structure: most code is shared across section types.

Production	Labeled BBs
section_header	68
ambiguous	202

Where the labels land. Figure 4 shows 12 of the 68 labeled blocks, taken directly from our out.dot. They cluster in three places: the harness’s walk_section_headers loop, libelf’s __libelf_set_rawdata_wrlock section-data initialization routine, and libelf’s lookup routines (elf_strptr, gelf_getsym). The cross-function arrows are real edges from the trace: control flows from the harness’s conditional branch into libelf’s section setup and from there into the symbol-table walk only for sections that actually carry a symbol table.

Accuracy. Manual inspection of all 68 labeled blocks yields 62 true positives (91.2%) and 5 false positives. The 5 false positives all sit inside __gelf_getehdr_rlock, an ELF-header validation routine that libelf calls internally during section traversal. Those blocks process the ELF header rather than section headers, but they are reached only through section-handling code paths and so appear in the differential edge set. We did not identify any false negatives: unlabeled blocks in the graph correspond

12 of the 68 basic blocks labeled section_header by ParseRE

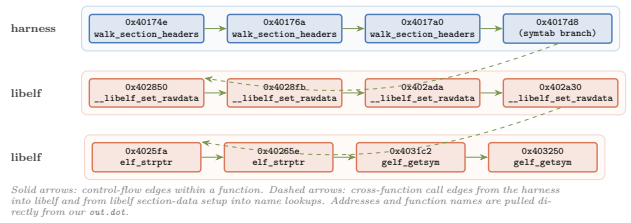


Figure 4. 12 of the 68 section_header-labeled basic blocks. Solid arrows are intra-function control flow; dashed arrows are cross-function call edges.

TABLE 4. Cross-format evaluation summary. Inst. = template instantiations; BBs = total basic blocks in the output CFG; Labels = blocks that received a label; Acc. = strict accuracy (TP / (TP + FP)).

Format	Parser	Inst.	BBs	Labels	Acc.(%)
URI	Curl	1,458	522	211	—
URI	Apache APR	1,458	—	—	—
JSON	json-c	27	464	294	—
HPACK	nghttp2	—	—	—	—
ELF	libelf	20	358	68	91.2

either to memory allocation, common initialization, or libelf’s type-uniform readers like gelf_getphdr.

What this reveals about binary formats. Text-format parsers label 6 to 11 distinct productions (JSON, URI). Our binary-format evaluation labels exactly one (section_header). This is not a defect of ParseRE; it accurately captures the structural property that binary parsers use uniform record readers for most structures and conditional consumer code for the rest. The labels show up where the conditional logic lives.

5.6. Cross-Format Comparison

Coverage is not a function of corpus size. JSON uses 27 instantiations and labels 294 blocks (63% of the graph). URI uses 1,458 instantiations and labels 211 blocks (40%). ELF uses 20 instantiations and labels 68 blocks (19%). The labeling rate tracks the structural complexity of the parser rather than the size of the input corpus. Text formats with per-production parser routines saturate with small corpora. Binary formats with uniform record readers produce sparse but high-confidence labels regardless of corpus size.

[BEN: fill in the remaining accuracy columns when Rishav’s labeled SVGs and your HPACK results arrive.]

6. Related Work

ParseRE occupies a specific niche in the binary analysis landscape: it transfers known input-format labels onto a binary’s basic blocks using only control-flow traces. We position it against three lines of prior

work that are easily confused with each other but solve materially different problems.

6.1. Trace-Diffing Tools

Tenet [3] and Lighthouse [8] are disassembler plugins that highlight the basic blocks differing between execution traces of the same binary inside IDA Pro or Binary Ninja. They are widely used in practice for tasks like identifying why one input crashes while another does not.

These tools solve a degenerate case of ParseRE’s problem: $N=2$ inputs, no input-side annotations. ParseRE generalizes the differential approach in two ways. First, we systematically generate large corpora of structurally related inputs from a grammar template (Section 3.1) rather than asking the user to supply two interesting inputs by hand. Second, we attach semantic labels to the input structure and propagate those labels to CFG edges, which produces a format-aware annotation rather than a binary “differs / does not differ” verdict.

6.2. Input-Region Categorization

PolyFile [1] from Trail of Bits identifies which byte ranges of a file belong to which substructure of a known file format. PolyTracker [2] extends this with whole-input dynamic information flow tracking, using LLVM instrumentation to build a directed acyclic graph that maps each input byte through the program’s execution.

ParseRE and these tools both operate on inputs whose formats are known, but they differ in what they annotate. PolyFile produces an annotation of the input. PolyTracker produces a data-flow mapping between input bytes and program operations. ParseRE produces an annotation of the binary’s control-flow graph itself. The three tools are therefore composable: PolyFile can produce the labeled corpus that ParseRE consumes, and ParseRE’s output can be cross-referenced with PolyTracker’s data-flow graph for deeper understanding.

A methodological distinction separates ParseRE from PolyTracker: PolyTracker requires LLVM-level compiler instrumentation of the target program, which restricts it to scenarios in which source code is available. ParseRE requires only QEMU user-mode emulation, which is available for any binary the analyst can run. That covers the reverse-engineering scenario in which source is unavailable.

6.3. Grammar Mining and Format Recovery

A substantial body of academic work addresses the inverse problem: given only a binary, recover the grammar of the inputs it accepts. Tupni [9] performs automatic reverse engineering of input formats using dynamic analysis. Polyglot [10] extracts protocol message

formats via dynamic binary analysis. AUTOGRAM [11] infers grammars from observed input-output behavior using dynamic taint analysis. More recent work by Gopinath et al. [12] mines context-free input grammars from dynamic control flow. Prospex [13] recovers protocol state machines from network traces.

These approaches answer the question “what does this binary eat?” ParseRE answers a different question: “given that this binary eats X, where in the code is X handled?” The two problems are sequential rather than competing. An analyst would use a grammar-recovery tool to obtain a starting grammar, then use ParseRE to map that grammar onto the code. Conflating the two would be a category error.

A second methodological distinction is fundamental. Most grammar-recovery approaches require some form of program instrumentation, whether compiler-level for data-flow tracing [2] or source-level for taint propagation [11]. That restricts them to scenarios in which source is available, which defeats the central reverse-engineering use case where the analyst typically has only a binary. ParseRE requires only QEMU-traceable control flow, which is available for any binary.

6.4. Differential Testing for Parsers

The broader idea of detecting parser bugs through differential observation has been explored in the HTTP/1.1 ecosystem [14] and in static differential analysis of protocol parsers [15]. These tools surface discrepancies in protocol semantics between parser implementations. ParseRE surfaces the structural correspondence between an agreed input grammar and the code that realizes it. They are complementary: a differential-testing tool can find that two parsers disagree, then ParseRE can localize where in each parser the disagreement lives.

[BEN: confirm whether the HTTP Garden citation should be included in the camera-ready (and excluded from the anonymous submission to avoid identity leakage). Currently we cite T-Reqs and ParDiff instead.]

7. Future Work

Additional architectures. ParseRE currently targets x86-64 binaries. QEMU supports many additional architectures (ARM, MIPS, RISC-V, PowerPC), and the only architecture-dependent component is the trace-log parser. Extending to other architectures requires adapting the regular expression that extracts basic-block boundaries from QEMU’s log output.

Differential fuzzing guided by labels. Once ParseRE has labeled a parser’s CFG, the labels provide a natural coverage metric for format-aware fuzzing. A fuzzer could prioritize inputs that exercise under-explored grammar productions, using ParseRE’s labels as a coverage map.

Grammar mining integration. As discussed in Section 6.3, grammar-recovery tools and ParseRE solve

complementary problems. An integrated pipeline could first recover an approximate grammar from a binary, then use ParseRE to validate and refine the mapping, closing the loop between “what does this binary eat?” and “where does it eat it?”

Hierarchical labeling. The current template system supports one level of production-to-label mapping. Extending to hierarchical templates (labeling not just “string” but “string-escape-sequence” versus “string-literal-characters”) would increase the granularity of the output without algorithmic changes.

8. Ethical Considerations

ParseRE is a defensive reverse-engineering tool that annotates binary code with format-semantic labels. It does not discover, create, or exploit vulnerabilities. All target parsers used in our evaluation are open-source software analyzed with the knowledge and consent of the authors. No proprietary or classified binaries were analyzed.

9. LLM Usage Statement

Large language models were used for editorial purposes in preparing this manuscript. All outputs were inspected by the authors to ensure accuracy and originality. No LLM was used in the design, implementation, or evaluation of ParseRE itself.

10. Conclusion

We presented ParseRE, a technique for transferring known input-format labels onto the control-flow graph of an unknown binary. By systematically generating structurally varied inputs, tracing them under QEMU, and attributing control-flow deltas to parse-tree deltas, ParseRE produces a semantically meaningful map of parser code without requiring source code, compiler instrumentation, or data-flow analysis.

We evaluated ParseRE on four formats spanning the language hierarchy (URI, JSON, HPACK, and ELF), demonstrating high accuracy through exhaustive manual inspection of every label. ParseRE is fast, lightweight, and integrates directly into existing reverse-engineering workflows.

[BEN: final accuracy summary sentence pending. Once Rishav’s labels and your HPACK numbers are in, we can say something like “Across all four formats, ParseRE achieves an average accuracy of X%, with trace-collection times under Y seconds per format.”]

Acknowledgment

References

- [1] E. Sultanik, “PolyFile: A polyglot-aware file identification and mapping utility,” <https://github.com/trailofbits/polyfile>, 2019, trail of Bits; funded by DARPA SafeDocs.

- [2] E. Sultanik, M. Surovič, H. Brodin, K. Kaoudis, F. Tiesca, C. Harmon, L. Overall, J. Sweeney, and B. Larsen, “PolyTracker: Whole-input dynamic information flow tracing,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1841–1845.
- [3] M. Gaasedelen, “Tenet: A trace explorer for reverse engineers,” <https://github.com/gaasedelen/tenet>, 2021, IDA Pro plugin; 1st place, Hex-Rays Plugin Contest 2021.
- [4] N. Chomsky, “Three models for the description of language,” *IRE Transactions on Information Theory*, vol. 2, no. 3, pp. 113–124, 1956.
- [5] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel et al., “SoK: (state of) the art of war: Offensive techniques in binary analysis,” in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 138–157.
- [6] F. Bellard, “QEMU, a fast and portable dynamic translator,” in *Proceedings of the 2005 USENIX Annual Technical Conference, FREENIX Track*. Anaheim, CA: USENIX Association, 2005, pp. 41–46.
- [7] R. Peon and H. Ruellan, “HPACK: Header compression for HTTP/2,” RFC 7541, 2015.
- [8] M. Gaasedelen, “Lighthouse: A code coverage explorer for reverse engineers,” <https://github.com/gaasedelen/lighthouse>, 2017, IDA Pro / Binary Ninja plugin; 2nd place, Hex-Rays Plugin Contest 2017.
- [9] W. Cui, M. Peinado, K. Chen, H. J. Wang, and L. Irun-Briz, “Tupni: Automatic reverse engineering of input formats,” in *Proceedings of the 15th ACM Conference on Computer and Communications Security*, ser. CCS ’08. Association for Computing Machinery, 2008, pp. 391–402.
- [10] J. Caballero, H. Yin, Z. Liang, and D. Song, “Polyglot: Automatic extraction of protocol message format using dynamic binary analysis,” in *Proceedings of the 14th ACM Conference on Computer and Communications Security*, 2007, pp. 317–329.
- [11] M. Höschle and A. Zeller, “Mining input grammars from dynamic taints,” in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 720–725.
- [12] R. Gopinath, B. Mathis, and A. Zeller, “Mining input grammars from dynamic control flow,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 172–183.
- [13] P. M. Comparetti, G. Wondracek, C. Kruegel, and E. Kirda, “Prospex: Protocol specification extraction,” in *Proceedings of the 30th IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2009, pp. 110–125.
- [14] B. Jabiyeve, S. Sprecher, K. Onarlioglu, and E. Kirda, “T-Reqs: HTTP request smuggling with differential fuzzing,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’21. Association for Computing Machinery, 2021, pp. 1805–1820.
- [15] M. Zheng, Q. Shi, X. Liu, X. Xu, L. Yu, C. Liu, G. Wei, and X. Zhang, “ParDiff: Practical static differential analysis of network protocol parsers,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 1208–1234, 2024.